

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY



PLAYWRIGHT LOCATORS

Complete Guide with Examples

Date: _____



What are Locators in Playwright?

- Locators are strategies to find elements on a web page
- Playwright has built-in smart locators (better than Selenium!)
- Locators are strict by default - must match exactly one element
- Locators auto-wait for elements to be ready!

! KEY DIFFERENCE!

Playwright vs Selenium:

Selenium: Locators find element immediately (can fail)

Playwright: Locators are lazy - they wait & retry automatically!

Playwright locators are resilient and handle timing issues automatically! ✨



Sample HTML for Examples

```
<form id="login-form"> <label for="email">Email:</label> <input
type="email" id="email" placeholder="Enter email" data-testid="email-
input" /> <label for="password">Password:</label> <input
type="password" id="password" placeholder="Enter password" /> <button
type="submit" class="btn primary">Login</button> <button
type="button">Cancel</button> <a href="/forgot">Forgot Password?</a>
<p title="Help text">Need help? Contact support</p> </form>
```

🌟 Recommended Locators (Use These!)

1. GETBYROLE() 🌟

Find by **ARIA role** - Most accessible & recommended!

{ } getByRole() Examples

```
import { test } from '@playwright/test';
test('getByRole examples', async ({ page }) => {
  // Find button by role
  await page.getByRole('button').click();
  // Find specific button by name (visible text)
  await page.getByRole('button', { name: 'Login' }).click();
  // Find link
  await page.getByRole('link', { name: 'Forgot Password?' }).click();
  // Find textbox (input)
  await page.getByRole('textbox', { name: 'Email'
}).fill('test@test.com');
  // Common roles: button, link, textbox, checkbox, radio, heading
  await page.getByRole('heading', { name: 'Welcome' });
  await page.getByRole('checkbox', { name: 'Remember me' }).check();
});
```

💡 **WHY GETBYROLE() IS BEST:**

- ✓ *Accessible* - Works with screen readers
- ✓ *Semantic* - Describes what element does
- ✓ *Resilient* - Not affected by CSS/ID changes
- ✓ *Readable* - Clear what you're testing

Always prefer this over CSS or XPath when possible!

2. GETBYLABEL() ★

Find input by its associated label text

{ } getByLabel() Examples

```
test('getByLabel examples', async ({ page }) => {  
  // Find input by its label  
  await page.getByLabel('Email').fill('user@test.com');  
  await page.getByLabel('Password').fill('secret123');  
  // Exact match (case-sensitive)  
  await page.getByLabel('Email:', { exact: true }).fill('test@test.com');  
  // Works with checkbox/radio labels too  
  await page.getByLabel('I agree to terms').check();  
});
```

3. GETBYPLACEHOLDER() ★

Find input by placeholder text

{ } getByPlaceholder() Examples

```
test('getByPlaceholder examples', async ({ page }) => {  
  // Find by placeholder  
  await page.getByPlaceholder('Enter email').fill('test@test.com');  
  await page.getByPlaceholder('Enter password').fill('pass123');  
  // Partial match  
  await page.getByPlaceholder('email').fill('user@test.com');  
  // Search box example
```

```
await page.getByPlaceholder('Search products...').fill('laptop');
});
```

4. GETBYTEXT() ★

Find element by its **visible text content**

{ } getByText() Examples

```
test('getByText examples', async ({ page }) => {
  // Find by exact text
  await page.getByText('Login').click();
  // Find by partial text
  await page.getByText('Forgot').click(); // Matches "Forgot Password?"
  // Exact match only
  await page.getByText('Login', { exact: true }).click();
  // Case-insensitive search
  await page.getByText('login').click(); // Finds "Login"
  // Using regex
  await page.getByText(/log.*in/i).click();
  // Find paragraph with text
  await page.getByText('Need help?').isVisible();
});
```

5. GETBYTESTID() ★

Find by **data-testid** attribute - stable & reliable!

{ } getByTestId() Examples

```
test('getByTestId examples', async ({ page }) => {
  // Find by data-testid attribute
  await page.getByTestId('email-input').fill('test@test.com');
  await page.getByTestId('submit-btn').click();
  // Best practice: Add data-testid to critical elements
  // HTML: <button data-testid="checkout-btn">Checkout</button>
```

```
await page.getByTestId('checkout-btn').click();
});
```

💡 BEST PRACTICES:

Priority Order (use in this order):

- 1 `getByRole()` - Most semantic & accessible
- 2 `getByLabel()` - Great for form inputs
- 3 `getByPlaceholder()` - Good for inputs
- 4 `getByText()` - Useful for unique text
- 5 `getByTestId()` - Reliable fallback
- 6 `CSS/XPath` - Last resort only!

6. GETBYTITLE()

Find by `title attribute`

{ } getByTitle() Examples

```
test('getByTitle examples', async ({ page }) => {
  // Find by title attribute
  await page.getByTitle('Help text').click();
  // Useful for tooltips or icons with title
  // HTML: 
  await page.getByTitle('Profile Picture').click();
});
```

7. GETBYALTTEXT()

Find images by `alt attribute`

{ } getByAltText() Examples

```
test('getByAltText examples', async ({ page }) => {  
  // Find image by alt text  
  // HTML:   
  await page.getByAltText('Company Logo').click();  
  // Check if image is visible  
  await page.getByAltText('Product Image').isVisible();  
  // Clickable image  
  await page.getByAltText('Search Icon').click();  
});
```



CSS + XPath Locators (Use sparingly!)

8. PAGE.LOCATOR() - CSS SELECTOR

Generic locator using **CSS selectors**

{ } CSS Locator Examples

```
test('CSS locator examples', async ({ page }) => {  
  // By ID  
  await page.locator('#email').fill('test@test.com');  
  // By class  
  await page.locator('.primary').click();  
  // By attribute  
  await page.locator('[type="email"]').fill('user@test.com');  
  // Multiple classes  
  await page.locator('.btn.primary').click();  
  // Child selector  
  await page.locator('form > button').click();  
  // nth-child  
  await page.locator('button:nth-child(1)').click();  
});
```

9. PAGE.LOCATOR() - XPATH

Using XPath expressions

{ } XPath Locator Examples

```
test('XPath locator examples', async ({ page }) => {  
  // By ID  
  await page.locator('xpath=//input[@id="email"]').fill('test@test.com');  
  // By text  
  await page.locator('xpath=//button[text()="Login"]').click();  
  // Contains text  
  await page.locator('xpath=//a[contains(text(), "Forgot")]').click();  
  // Parent navigation  
  await page.locator('xpath=//input[@id="email"]/parent::form');  
  // Following sibling  
  await page.locator('xpath=//input[@id="email"]/following-sibling::button').click();  
});
```

Chaining & Filtering Locators

{ } Advanced Locator Techniques

```
test('chaining locators', async ({ page }) => {  
  // 1. Filter locators  
  await page  
    .locator('button')  
    .filter({ hasText: 'Login' })  
    .click();  
  // 2. Find within another locator  
  const form = page.locator('#login-form');  
  await form.getByLabel('Email').fill('test@test.com');  
  await form.getByRole('button').click();  
  // 3. Get by index (first, last, nth)  
  await page.getByRole('button').first().click();  
  await page.getByRole('button').last().click();  
  await page.getByRole('button').nth(1).click(); // 2nd button (0-indexed)
```

```
// 4. Count elements
const count = await page.getByRole('button').count();
console.log('Total buttons:', count);

// 5. Filter by text
await page
  .locator('div')
  .filter({ hasText: 'Welcome' })
  .click();

// 6. Filter by another locator inside
await page
  .locator('article')
  .filter({ has: page.getByRole('heading', { name: 'News' }) })
  .click();
});
```

Working with Multiple Elements

{ } Handle Lists & Multiple Elements

```
test('multiple elements', async ({ page }) => {
  // Get all matching elements
  const buttons = page.getByRole('button');
  // Count elements
  const count = await buttons.count();
  console.log('Total buttons:', count);
  // Loop through all elements
  for (let i = 0; i < count; i++) {
    const text = await buttons.nth(i).textContent();
    console.log(`Button ${i}: `, text);
  }
  // Get all text contents at once
  const allTexts = await buttons.allTextContents();
  console.log('All button texts:', allTexts);
  // Click all buttons one by one
  const links = page.getByRole('link');
  for (let i = 0; i < await links.count(); i++) {
    await links.nth(i).click();
    await page.goBack(); // Navigate back
  }
});
```


✓ Locator Assertions

{ } Common Locator Assertions

```
import { test, expect } from '@playwright/test';
test('locator assertions', async ({ page }) => {
  const button = page.getByRole('button', { name: 'Login' });
  // Check if visible
  await expect(button).toBeVisible();
  // Check if enabled
  await expect(button).toBeEnabled();
  // Check text content
  await expect(button).toHaveText('Login');
  // Check if has class
  await expect(button).toHaveClass(/primary/);
  // Check attribute value
  await expect(button).toHaveAttribute('type', 'submit');
  // Check if hidden
  await expect(page.locator('.error')).toBeHidden();
  // Check count
  await expect(page.getByRole('button')).toHaveCount(2);
  // Check if contains text
  await expect(page.getByText('Welcome')).toContainText('Welcome');
  // Check input value
  await expect(page.getByLabel('Email')).toHaveValue('test@test.com');
});
```

💡 PRO TIPS:

1. Always use `getByRole()` as first choice
2. Add `data-testid` attributes for critical elements
3. Avoid brittle selectors (CSS classes that change often)
4. Use `filter()` and `has()` for precise targeting
5. Locators are lazy - they don't find elements until an action is performed
6. Use strict mode - one locator should match one element

✗ COMMON MISTAKES:

1. Using weak locators:

❌ Bad: `page.locator('button')` (too generic)

✅ Good: `page.getByRole('button', { name: 'Submit' })`

2. Not handling multiple matches:

→ Use `.first()`, `.last()`, `.nth()` when needed

3. Relying on CSS classes that change:

→ Classes often change with styling updates

→ Use semantic locators instead

4. Forgetting to await:

❌ Bad: `page.locator('button').click()`

✅ Good: `await page.locator('button').click()`



PRACTICE EXERCISE:

Task: Practice different locator strategies

Go to: <https://demo.playwright.dev/todomvc/>

1. Add a todo using `getByPlaceholder()`
2. Mark todo as complete using `getByLabel()`
3. Delete a todo using `getByRole('button')`
4. Count total todos using `.count()`
5. Check if "All" filter is active using `.toHaveClass()`
6. Clear completed todos using `getByRole()`

Bonus: Create a test that adds 5 todos, marks 3 as complete, and verifies counts!

KEY TAKEAWAYS:

- ✓ Playwright has 9+ built-in locator strategies
- ✓ `getByRole()` is the most recommended approach
- ✓ Locators are lazy & auto-wait for elements
- ✓ Use semantic locators over CSS/XPath
- ✓ Add `data-testid` for critical elements
- ✓ Chain & filter locators for precision
- ✓ Use `.first()`, `.last()`, `.nth()` for multiple matches
- ✓ Always await locator actions!

Happy Testing! 🚀

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY